

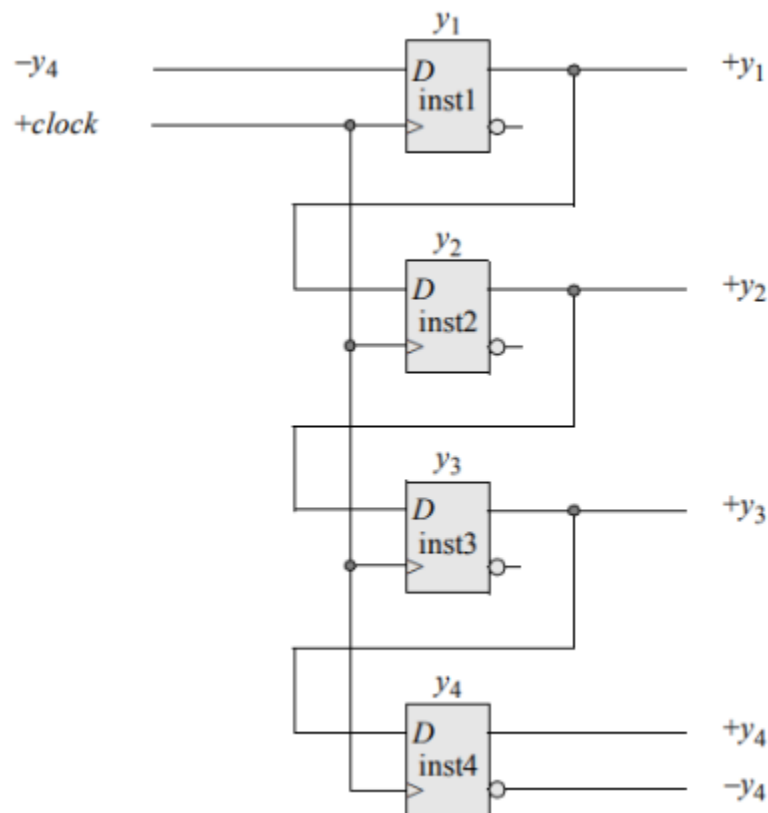
Assignment-05: User Defined Primitives

Combinational UDP

- Full Subtractor
- Three input majority function
- Even Parity Generator
- Custom Logic function (output =1 when input 000,110 and 111 else output=0)
- 4-bit binary to gray code converter
- 8 : 3 Encoder
- 2 bit adder cum subtractor

Sequential UDP

- Design Johnson Counter using **User Defined** Edge triggered D-FF.



Use following edge triggered D-FF

```

//a positive-edge-sensitive D flip-flop
primitive udp_dff_edgel (q, d, clk, rst_n);

input d, clk, rst_n;
output q;

reg q;      //q is internal storage

//initialize q to 0
initial
    q = 0;                                //continued on next page
//define state table
table
//inputs are in the same order as the input list
// d      clk  rst_n : q : q+;  q+ is the next state
    0      (01) 1    : ? : 0;    //(01) is rising edge
    1      (01) 1    : ? : 1;    //rst_n = 1 means no rst
    1      (0x) 1    : 1 : 1;    //(0x) is no change
    0      (0x) 1    : 0 : 0;
    ?      (?0) 1    : ? : -;    //ignore negative edge
//reset case when rst_n is 0 and clk has any transition
    ?      (??) 0    : ? : 0;    //rst_n = 0 means reset
//reset case when rst_n is 0. d & clk can be anything, q+=0
    ?      ?      0    : ? : 0;
//reset case when 0 --> 1 transition on rst_n. Hold q+ state
    ?      ?      (01) : ? : -;
//non-reset case when d has any trans, but clk has no trans
    (??) ?      1    : ? : -;    //clk = ?, means no edge
endtable
endprimitive

```

Testbench for checking the Johnson counter

```

//test bench for the 4-bit johnson counter
module ctr_johnson4_tb;

reg clk, rst_n;           //inputs are reg for tb
wire y1, y2, y3, y4;      //outputs are wire for tb

initial
$monitor ("count = %b", {y1, y2, y3, y4});

initial                    //define clk
begin
    clk = 1'b0;
    forever
        #10    clk = ~clk;
end

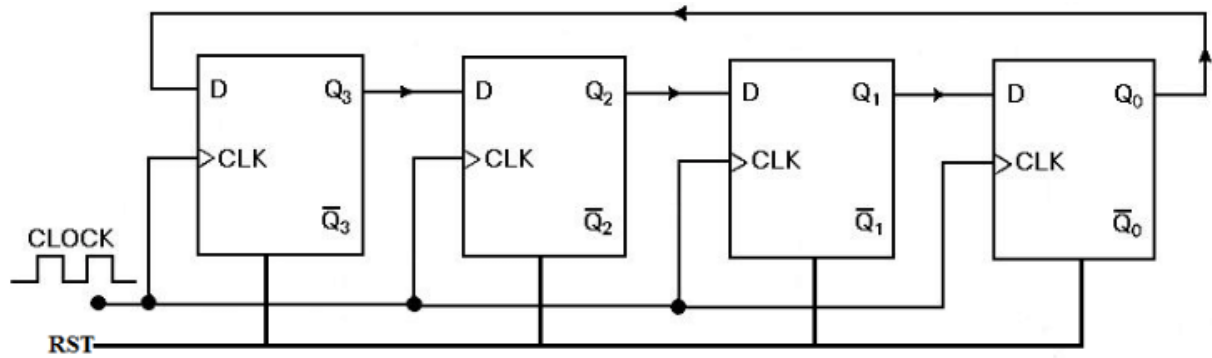
initial                    //define reset
begin
    #0 rst_n = 1'b0;
    #5 rst_n = 1'b1;
    #200 $stop;
end

ctr_johnson4 inst1 (        //instantiate the module
    .rst_n(rst_n),
    .clk(clk),
    .y1(y1),
    .y2(y2),
    .y3(y3),
    .y4(y4)
    );
endmodule

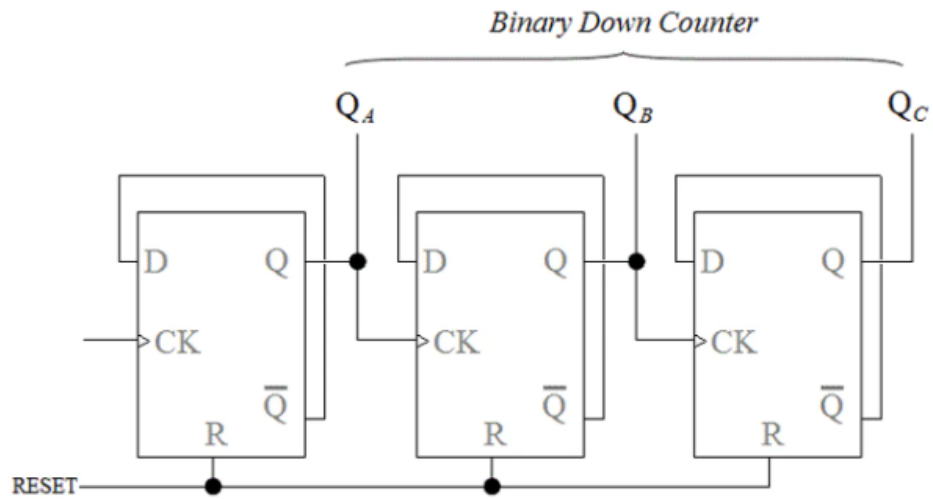
```

Solve the following using above reference

Ring Counter



3-Bit down counter



Shift Register

